

Java Assessment

The Brief

Build a small internal Copilot for a specific professional domain (assigned to you below). It ingests a document corpus, answers questions about it with **real citations**, refuses to answer when it shouldn't, and carries out a short multi-step task through a small team of AI agents — **with a person required to approve the consequential step before it executes**.

Your Assignment

Told to you directly — domain and twist, one each, not derived or swapped.

Domains: Legal contract review · Curriculum design · Government citizen services · HR screening

Twists: Cost governor · Audit & replay · Bilingual retrieval (AR+EN) · Human review queue · multi-tenancy (2 tenants)

Requirements

1. Ingestion

- At least **2 different input formats** (e.g. PDF and Markdown, or PDF and DOCX) — not two files of the same format
- A visible pipeline with distinct stages: extract → clean → chunk → embed → index. Each stage should be a separate method/class you could point to, not one function doing everything
- Every stored chunk carries metadata: source document, section/page, and a version or ingestion timestamp
- Re-running ingestion on the same document must not duplicate it — this needs to work, not just be claimed
- **Corpus size:** 15–20 documents, 60–80 pages total, public or synthetic.

2. Retrieval

- State your chunking strategy (fixed-size, semantic, structural) and **why it fits your document type**

- **Hybrid retrieval is required:** dense (embedding) search combined with keyword search, with a documented method for combining the two results (e.g. reciprocal rank fusion, weighted score merge)
- One additional retrieval enhancement of your choice: re-ranking, query rewriting, HyDE, multi-query expansion, or metadata filtering
- Every answer must cite the exact chunk(s) it came from — document ID, section/page, not just "source: corpus"
- **Refusal must be measurable, not vibes-based:** define a concrete threshold or rule for when the system says, "not enough information in the corpus," and show it firing correctly on out-of-corpus questions in your evaluation set

3. Evaluation — the part most people skip

- A golden set of **at least 15 question/answer pairs**, including **at least 3 adversarial cases**: one out-of-corpus question, one ambiguous question, one prompt-injection attempt embedded in a document
- A runnable script or test that reports, with actual numbers: retrieval hit-rate, groundedness (did the answer match the cited chunk), and refusal correctness (did it refuse when it should have)
- Report your real numbers, including the bad ones, with one or two sentences of interpretation. A eval harness that always shows 100% is a red flag, not a good sign

4. Multi-agent workflow — specific roles, not one prompt wearing different hats

- **At least 3 specialised agents plus an orchestrator.** Each agent needs: a name, a stated single responsibility, a restricted list of tools it's allowed to call, and a defined termination condition (what tells it "I'm done")
- **At least 4 distinct tools total**, of which **at least 1 is write/side-effecting** — this tool must be schema-validated before execution and must never run without passing the approval gate
- Agents talk to each other through a defined, typed structure, not by-passing raw strings back and forth.
- The orchestration pattern must be named and justified (supervisor, pipeline, planner-executor, state machine) — pick one on purpose, don't default into one by accident
- Include: a max-iteration limit so an agent loop can't run forever, a per-step timeout, and a fallback to plain retrieval-and-answer if the agent workflow fails

5. Human approval gate

- At least one step in the workflow must stop and wait for a person to approve, reject, or edit-and-approve before anything happens
- All three outcomes (approve/reject/edit) must be recorded — who decided, when, and what changed if edited

6. Observability

- A correlation/run ID that follows a single request from the API layer through the orchestrator, through each agent, to each LLM call — you should be able to pull every log line for one run by that ID alone
- Per-request token usage and estimated cost, stored somewhere queryable (a table is fine — it doesn't need a dashboard)
- Basic health/readiness endpoints (Actuator is sufficient)

7. Security

- Real authentication, **2 roles with genuinely different, server-enforced permissions**
- Parameterised queries throughout — no string-concatenated SQL, anywhere
- At least **2 prompt-injection test cases** in your evaluation set, at least one of them embedded *inside an ingested document* (the indirect case), and your system must demonstrably resist it — show the resistance, don't just claim it

8. Documentation

Put these in your README.md

- **Chunking strategy** — which one you picked (fixed-size, semantic, structural) and *why it fits your specific document type*
- **Retrieval fusion method** — how you combined dense and keyword results, in a sentence or two
- **Which additional retrieval enhancement** you chose and why
- **Setup instructions** — how to run the project, how to run tests, and a numbered demo script.
- **Where to find token usage/cost per request** — note where it's stored and how to query it (a table is fine — no dashboard needed)
- **One-sentence orchestration summary** — which pattern you used (supervisor, pipeline, planner-executor, state machine) and why.

